

CS425 - Software Architecture

LAB 6

Java Development Environment and Coding Practice Exercises

Student Records Management Command-Line Application

Prepared by: Samson Kitigo

Student ID: 618666

Date: August 4, 2026

Implementation: Java 8-compatible source and bytecode; verified on installed JDK 26

Submission package: A complete local Git repository with production source, dependency-free tests, actual terminal transcripts, rendered evidence screenshots, and this report.

Delivered Components

- Student domain model with all required fields, three constructors, getters, and setters.
- Executable student-records application with required sorting and Platinum Alumni logic.
- HelloWorld and second-biggest exercises, plus 13 automated verification checks.
- Evidence images generated from actual captured command output.

1. Assignment Coverage

The implementation maps each Lab 6 requirement to a concrete source file or verified result.

Requirement	Implementation evidence	Status
Student model	Student.java: fields, three constructors, accessors, mutators	Complete
Required packages	edu.mum.cs.cs425.demos.studentrecordsmgmtapp and .model	Complete
Ascending student names	printListOfStudents copies and sorts the array by name	Complete
Platinum Alumni	30-year cutoff; admission date descending	Complete
printHelloWorld	Handles multiples of 5, 7, and both	Complete
findSecondBiggest	One-pass, second distinct maximum; no sorting	Complete
Execution evidence	Four transcripts and screenshots in evidence/	Complete
Git repository	Initialized and committed locally; remote publication pending	Prepared

Project Structure

```
src/main/java/.../model/Student.java
src/main/java/.../MyStudentRecordsMgmtApp.java
src/test/java/.../Lab6SelfTest.java
build.sh | run.sh | test.sh | capture-evidence.sh
evidence/ | docs/ | README.md
```

Build and Execution

```
./build.sh
./run.sh all
./test.sh
./capture-evidence.sh
```

Compatibility: The build uses javac --release 8. javap reports class-file major version 52, which is the Java 8 bytecode level.

2. Java Development Environment

Item	Verified value
Active runtime/compiler	Oracle Java 26.0.2
Additional installed JDK	OpenJDK 21.0.9
Target compatibility	Java 8 source/bytecode via --release 8
IDE/terminal requirement	Implemented and executed as a command-line Java application

Environment note: The supplied assignment names legacy JDK 8, 11, and 13. This machine currently has JDK 21 and 26, so the evidence below reports the installed versions truthfully. The application is nevertheless compiled to Java 8-compatible bytecode.



```

Terminal - jdk-environment.txt

$ java -version
java version "26.0.2" 2026-07-21
Java(TM) SE Runtime Environment (build 26.0.2+10-55)
Java HotSpot(TM) 64-Bit Server VM (build 26.0.2+10-55, mixed mode, sharing)

$ javac -version
javac 26.0.2

$ /usr/libexec/java_home -V
Matching Java Virtual Machines (2):
  26.0.2 (arm64) "Oracle Corporation" - "Java SE 26.0.2"
/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home
  21.0.9 (arm64) "Homebrew" - "OpenJDK 21.0.9"
/opt/homebrew/Cellar/openjdk@21/21.0.9/libexec/openjdk.jdk/Contents/Home
/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home

```

Figure 1. Actual JDK and compiler environment output.

3. Student Records Management

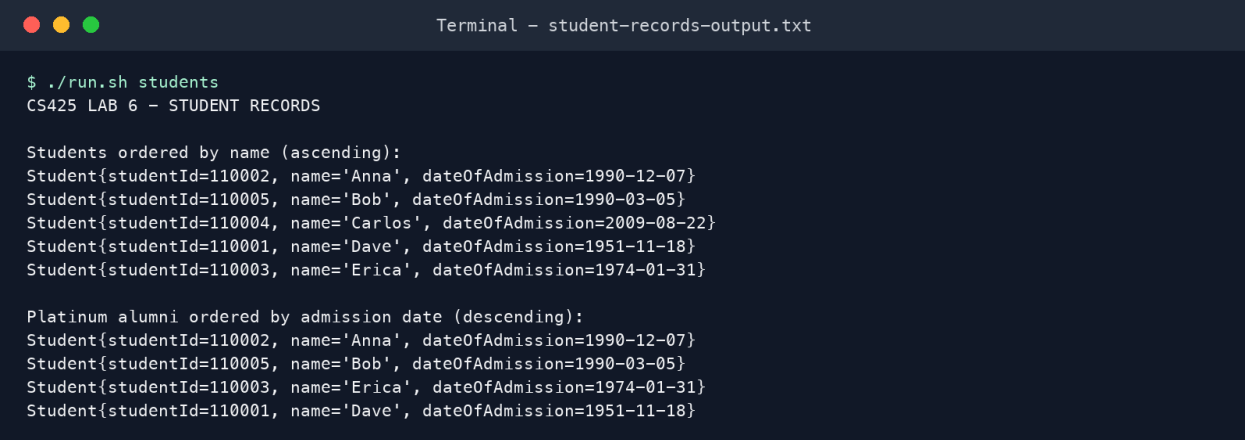
Student responsibilities. The model retains `studentId`, `name`, and `dateOfAdmission`. It provides the default constructor, a `LocalDate` constructor, a convenience `MM/dd/yyyy` constructor, and all required accessors and mutators.

```
public class Student {
    private long studentId;
    private String name;
    private LocalDate dateOfAdmission;

    public Student() { }
    public Student(long id, String name, LocalDate admitted) { ... }
    public Student(long id, String name, String admitted) { ... }
}
```

Ordering rules. `printListOfStudents` orders a defensive copy by name ascending.

`getListOfPlatinumAlumniStudents` selects admission dates on or before the date 30 years ago, then orders the result by admission date descending.



```
Terminal - student-records-output.txt

$ ./run.sh students
CS425 LAB 6 - STUDENT RECORDS

Students ordered by name (ascending):
Student{studentId=110002, name='Anna', dateOfAdmission=1990-12-07}
Student{studentId=110005, name='Bob', dateOfAdmission=1990-03-05}
Student{studentId=110004, name='Carlos', dateOfAdmission=2009-08-22}
Student{studentId=110001, name='Dave', dateOfAdmission=1951-11-18}
Student{studentId=110003, name='Erica', dateOfAdmission=1974-01-31}

Platinum alumni ordered by admission date (descending):
Student{studentId=110002, name='Anna', dateOfAdmission=1990-12-07}
Student{studentId=110005, name='Bob', dateOfAdmission=1990-03-05}
Student{studentId=110003, name='Erica', dateOfAdmission=1974-01-31}
Student{studentId=110001, name='Dave', dateOfAdmission=1951-11-18}
```

Figure 2. Actual student sorting and Platinum Alumni output.

4. Coding Practice Exercises

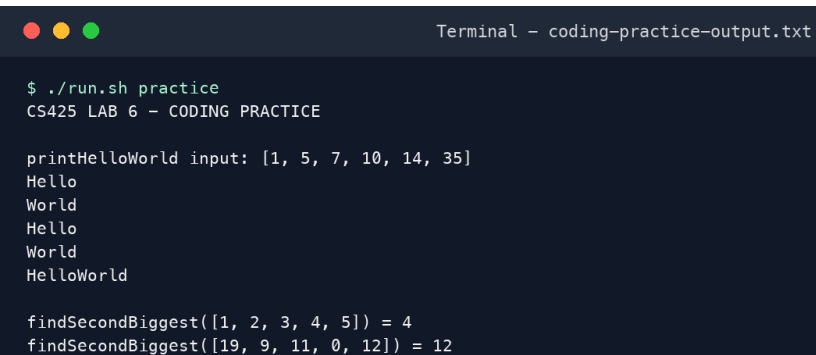
printHelloWorld

Each integer is evaluated once. Multiples of both 5 and 7 are checked first so 35 produces HelloWorld instead of two separate outputs. Multiples of only 5 produce Hello; multiples of only 7 produce World.

findSecondBiggest

The method tracks the largest and second-largest distinct values during a single pass. It does not sort the input array, uses $O(n)$ time and $O(1)$ auxiliary space, and rejects inputs without two distinct integers.

```
if (biggest == null || number > biggest) {
    secondBiggest = biggest;
    biggest = number;
} else if (number < biggest
    && (secondBiggest == null || number > secondBiggest)) {
    secondBiggest = number;
}
```



```
Terminal - coding-practice-output.txt

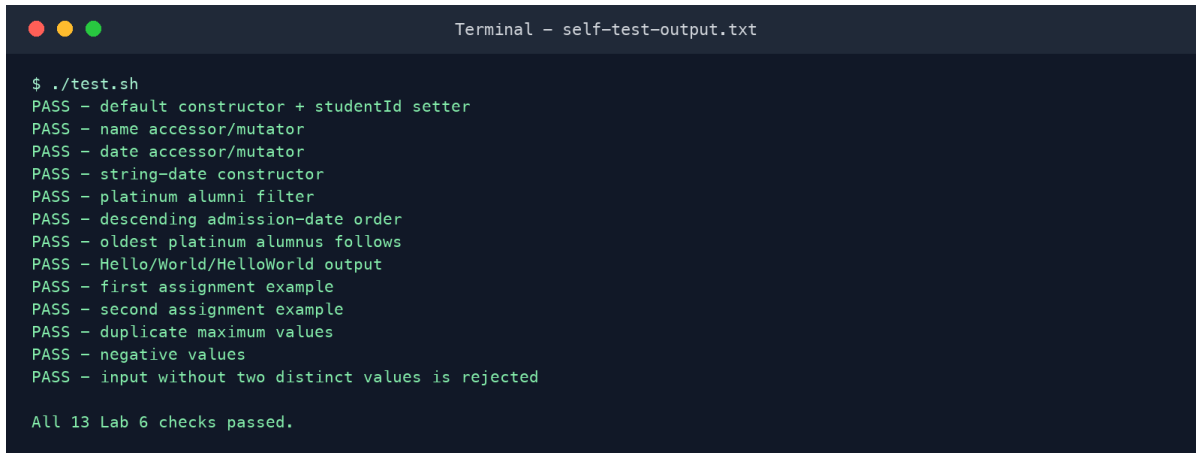
$ ./run.sh practice
CS425 LAB 6 - CODING PRACTICE

printHelloWorld input: [1, 5, 7, 10, 14, 35]
Hello
World
Hello
World
HelloWorld

findSecondBiggest([1, 2, 3, 4, 5]) = 4
findSecondBiggest([19, 9, 11, 0, 12]) = 12
```

Figure 3. Actual HelloWorld and second-biggest results.

5. Verification and Repository Handoff



```

Terminal - self-test-output.txt

$ ./test.sh
PASS - default constructor + studentId setter
PASS - name accessor/mutator
PASS - date accessor/mutator
PASS - string-date constructor
PASS - platinum alumni filter
PASS - descending admission-date order
PASS - oldest platinum alumnus follows
PASS - Hello/World/HelloWorld output
PASS - first assignment example
PASS - second assignment example
PASS - duplicate maximum values
PASS - negative values
PASS - input without two distinct values is rejected

All 13 Lab 6 checks passed.

```

Figure 4. Dependency-free verification: all 13 checks passed.

Validation	Result	Status
Compilation	javac --release 8	Complete
Bytecode level	Class major version 52 (Java 8)	Complete
Functional tests	13 of 13 checks passed	Complete
Required examples	Second biggest = 4 and 12	Complete
Repository state	Local git commit prepared	Complete
GitHub publication	Requires gh installation/authentication	Prepared

Publish and Submit

1. Install GitHub CLI if needed: `brew install gh`
2. Authenticate: `gh auth login`
3. From the repository directory, create and push the public repository: `gh repo create cs425-lab6-student-records-mgmt-app --public --source=. --remote=origin --push`
4. Submit the resulting GitHub repository URL in Sakai.

Publication status: The repository is fully prepared locally. Automatic GitHub publication was not performed because the required gh CLI is not installed or authenticated in this environment.